

**Prelude:** This computer assignment concerns the Block 5 (Estimation, Identification & Learning) of the course TRA455: Athletic Intelligence in Robotics (AIR) which includes Lectures 13, 14 & 15 and Tutorials 13, 14 & 15. It may also use the acquired knowledge from the previous blocks. You are encouraged to work in groups. Use of all aid is allowed. It is composed of 2 tasks.

**Submission Requirements:** We expect you to submit a **group** report in PDF format (preferably written in LaTeX) which explains all the solutions with the help of relevant plots as well as the Python code in a zip file. It is required to include a contribution statement at the end of your report where your group unanimously agree upon the extent of everyone's efforts in completing the assignment tasks. In order to pass the assignment, you need to achieve a minimum score of 40% of the total allocated points. The submission deadline is **27th May 2026**.

**Support:** If you need any help or further clarifications, please feel free to reach out to us.

- Mihaela Popescu (Task A): mihaela.popescu@dfki.de
- Sahand Wagemakers (Task B): sahand.wagemakers@chalmers.se
- Shivesh Kumar: shivesh.kumar@chalmers.se

## Task A: State Estimation (5P)

Consider the omnidirectional robot from Figure 1 with a state vector of the form  $\mathbf{x} = [x \ y \ \dot{x} \ \dot{y}]^T \in \mathbb{R}^4$ , where  $x$  and  $y$  are the position of the robot and  $\dot{x}$  and  $\dot{y}$  are the velocities. In this task, you will implement a Kalman filter to estimate the robot's state. For this, IMU ( $x, y$  acceleration) and GPS (latitude, longitude) data are synchronized and acquired at a frequency of 10 Hz. Random noise has been added to the IMU data for visualization purposes. The starting point for this exercise is the Python notebook provided on our Github here. Please solve the following tasks:

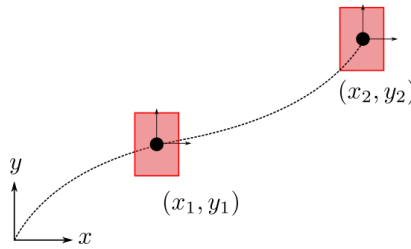


Figure 1: Omnidirectional robot with position  $x, y$  in plane.

1. Given the IMU acceleration input  $\mathbf{u} = [\ddot{x} \ \ddot{y}]^T$  and the GPS position update  $\mathbf{z} = [x \ y]^T$ , provide the discrete state space model for the Kalman filter prediction (matrices  $\mathbf{A}, \mathbf{B}$ ) and update (matrix  $\mathbf{H}$ ) equations with an integration time step of  $\Delta t = 0.1s$ . (Text answer) (1P)
2. Write the equations for the prediction and measurement update in the functions *kf\_predict* and *kf\_update*. (Python answer) (2P)
3. Define the state space matrices  $\mathbf{A}, \mathbf{B}, \mathbf{H}$  and the noise covariance matrices  $\mathbf{Q}, \mathbf{R}$ . (Python answer) (1P)

4. Call the prediction and update functions in the iterative *for*-loop. (Python answer) (0.5P)
5. Vary the process noise standard deviation  $std\_dev\_x$  and  $std\_dev\_y$  to larger and smaller values compared to  $std\_dev\_x\_gps$  and  $std\_dev\_y\_gps$ . What do you observe in the position tracking plots? (Text answer) (0.5P)

## Task B: System Identification (5P)

You are given the task to identify the mass-inertial parameters of the Unitree Go2 leg. Your colleague has conducted an experiment with the front left leg and sent you the data in `go2_leg_exp_500hz.csv` and `go2_leg_exp_1khz.csv` (Both are from the same experiment at different sampling rates). They have the following structure:

Table 1: CSV Data Structure

| column number(s) | column name | description                    | unit  |
|------------------|-------------|--------------------------------|-------|
| 0                | time        | timestamp                      | ms    |
| 1-3              | q0 - q2     | measured joint position vector | rad   |
| 4-6              | dq0 - dq2   | measured joint velocity vector | rad/s |
| 7-9              | tau0 - tau2 | measured joint torque vector   | N.m   |

The starting point for this exercise is the Python notebook provided on our Github here. A video of the experiment's lasting 45 seconds can also be viewed in the notebook (see Fig. 2 for a snapshot). You can watch it in the exercise notebook. Additionally, the notebook provides a code block for you to plot the joint data. Perform the system identification for the leg by completing the following tasks:

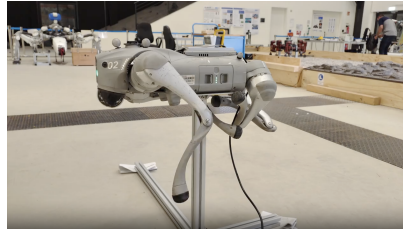


Figure 2: Data collection for system identification of the quadruped leg

1. Prepare the data for the identification. Use appropriate filters, compute missing data, and reject outliers if necessary. (1P)  
*Hint:* Use the `scipy` library for this task. You can append the filtered and computed signals to the dataframe. It is a good idea to name the new columns to avoid confusion.
2. Explain your choice of methods and discuss their advantages and disadvantages. (1P)
3. Obtain the regressor matrix using the `pinocchio` library. Note that the full matrix is constructed by vertically stacking the regressors from each time step. Additionally, you need to extract the joint torque vector for each time step from the dataframe and stack them vertically.

*Hint:* use the functions: `pin.computeJointTorqueRegressor`, `np.vstack` and `np.concatenate`.

Afterwards, estimate all model parameters using a **least squares regression** with the help of the **pseudo inverse**. (2P)

4. Validate your results. For that **print** the parameters you computed alongside the parameters from the URDF. Furthermore, **compute the joint torques that your new model predicts** and compare them with the measured ones by computing the torque root mean squared error (RMSE) and plotting them together. (1P)